DynaHug: Learning to Detect Malicious Pre-trained Models via Dynamic Analysis

***Abstract*—Pre-trained machine learning models (PTMs) are commonly provided via Model Hubs (e.g., Hugging Face) in standard formats like Pickles to facilitate accessibility and reuse. However, this ML supply chain setting is susceptible to malicious attacks that are capable of executing arbitrary code on trusted user environments, e.g., during model loading. To detect malicious PTMs, state-of-the-art detectors (e.g., PickleScan) rely on rules, heuristics, or static analysis, but ignore runtime model behaviors. Consequently, they either miss malicious models due to under-approximation (blacklisting) or miscategorize benign models due to over-approximation (static analysis or whitelisting). To address this challenge, we propose a novel technique (DynaHug) which detects malicious PTMs by learning the behavior of benign PTMs using dynamic analysis and machine learning (ML). DynaHug trains an ML classifier (one-class SVM (OCSVM)) on the runtime behaviours of task-specific benign models. We evaluate DynaHug using over 39,000 benign and malicious PTMs from different sources including Hugging Face and MalHug. We also compare DynaHug to several state-of-the-art detectors including static, dynamic and LLM-based detectors. Results show that DynaHug is up to 44% more effective than existing baselines in terms of F1-score. Our ablation study demonstrates that our design decisions (dynamic analysis, OCSVM, clustering) contribute positively to DynaHug's effectiveness.**

I. Introduction

Pre-trained machine learning models (PTMs) are typically provided via Model Hubs such as Hugging Face (HF) [1], Kaggle [2], GitHub [3], OpenCSG [4], SparkNLP [5] and ModelScope [6]. These PTM hosting platforms ease model accessibility and reuse; thereby supporting a large community of users. Practitioners and companies rely on Model Hubs for the distribution of their PTMs. Similarly, end users rely on third-party vendors to obtain PTMs. For instance, Hugging Face currently hosts over one (1) million ML artifacts and serves over 18.9 million visitors per month [7].

The huge reliance of the PTM ecosystem on third-party Model Hubs raises serious security concerns. Particularly, Model Hubs and end users are vulnerable to malicious actors. Attackers often upload malicious PTMs on Model Hubs to compromise user safety and system security [8]. This increases the risks of end-users executing malicious models in trusted execution environments (e.g., company servers and user's personal computers) [9], [10], [11], [12] [13], [14], [15], [16]. For instance, researchers have found malicious PTMs on Hugging Face that initiate a reverse shell connection to external servers, potentially allowing to send victim data to attacker-specified IP addresses [12].

To address this concern, malicious PTM detectors and scanners have been developed by Model Hubs and security practitioners. Hosting platforms, such as Hugging Face, employ scanners for the early detection and mitigation of malicious models [17], [18], [19]. For instance, Hugging Face's default security API employs four closed-source scanners using blacklisting, virus scanning, and static analysis [17], [18], [19].

Table I describes the characteristics of existing PTM scanners/detectors. On one hand, state-of-the-art (SOTA) detectors employ static analysis, heuristics, or blacklisting to detect malicious PTMs. For instance, PickleScan, ModelScan and HF PickleScan rely on blacklisted imports to detect malicious models [20], [21], [18]. However, these tools are ineffective in detecting previously unseen malicious payloads since blacklists are often non-exhaustive. Table II (last ("PyPI") column) illustrates how importing PyPI modules that support code execution (e.g., `execute` [22]) evades blacklisting methods (e.g., PickleScan and ModelScan [20], [21]). On the other hand, some approaches (e.g., ModelTracer [23]) employ dynamic analysis or blacklisting to detect malicious models. However, these approaches have high false positive rates – they *wrongly* flag benign models as malicious. More importantly, an attacker can easily evade a whitelist or blacklist set of rules, e.g., by using rare, new or previously unseen imports or system calls. Table II (second column) demonstrates how blacklisting methods (e.g., ModelTracer [23]) can lead to false positives.

To address this challenge, we propose an automated method (called DynaHug[1]) that learns the behavior of benign models using a combination of dynamic analysis and ML. To the best of our knowledge, DynaHug is the first technique that automatically learns to detect malicious models via dynamic behavioral analysis. Figure 2 and Algorithm 1 present the design and algorithm of our DynaHug approach. DynaHug learns the behaviors of benign models by first clustering models by tasks (e.g., `text-classification`). Secondly, it conducts dynamic analysis of the model in a sandbox and collects system call traces. In this step, PTMs are executed by loading the model and deserializing it. Next, it trains a one-class SVM that learns the behavior of benign models. At inference, when given the system call traces of a PTM under test (PUT), DynaHug automatically detects whether it is malicious or benign.

This work makes the following contributions:

1) **DynaHug**: We propose an automated method (called DynaHug) that learns to detect malicious PTMs by learning the behavior of task-specific clusters of benign models via dynamic program analysis. DynaHug provides an automated learning method for detecting malicious PTMs in Model Hubs such as Hugging Face.

[1] DynaHug means "**Dyna**mic **Hug**ging Face Malicious PTM Detector"

TABLE I: Details of state-of-the-art detectors versus our approach (DYNAHUG) showing whether the detector "fully" (●), "partially" (◐), or "does not" (○) employ the specified analysis technique.)

Detection Tools	Rule Based	Import Scanning	Blacklist	Whitelist	Heuristic	Dataflow Analysis	Vulnerability Detection	Restricted Loader	Machine Learning	Static Analysis	Dynamic Analysis	Open Source
PickleScan [20]	●	●	●	◐	○	○	○	○	○	●	○	●
ModelScan [21]	●	●	●	○	○	○	○	○	○	●	○	●
Fickling [24]	●	●	●	◐	○	●	○	○	○	●	○	●
MALHUG [8]	●	●	●	○	●	○	○	○	○	●	○	○
PickleBall [25]	◐	●	○	●	●	○	○	●	○	●	○	●
ModelTracer [23]	●	○	●	○	○	○	○	○	○	○	●	●
Weights-Only [26]	●	●	○	●	○	○	○	●	○	●	○	●
JFrog (HF) [19]	●	●	●	○	○	○	○	○	○	●	○	○
Guardian (HF) [17]	●	●	●	○	○	○	◐	○	○	●	○	○
ClamAV (HF) [18]	○	○	●	○	○	○	●	○	○	●	○	●
HF_PickleScan [18]	●	●	●	◐	○	○	○	○	○	●	○	○
SafePickle [27]	○	○	○	○	○	○	○	○	●	●	○	○
DYNAHUG	○	○	○	○	○	○	○	○	●	○	●	●

2) **Evaluation**: We evaluate DYNAHUG using over 39,000 PTMs. Our experiments demonstrate that DYNAHUG is effective in detecting malicious models with an F1-score of up to 0.9963 across all datasets.
3) **State-of-the-art Comparison:** We compare DYNAHUG to six (6) PTM detectors – PickleScan, ModelScan, Fickling, ModelTracer and LLM-based detectors (Llama-3.1-8B-Instruct-tuned and GPT-5.2). Results show that DYNAHUG is up to 44% more effective than the baselines in terms of F1-score.
4) **Ablation and Sensitivity Studies:** We report ablation studies examining whether our design decisions (e.g., dynamic analysis, etc.) contribute positively to DYNAHUG's effectiveness and outperform alternative design choices. Additionally, we conduct probing and sensitivity studies examining the impact of task clustering, and training sizes on DYNAHUG's performance.

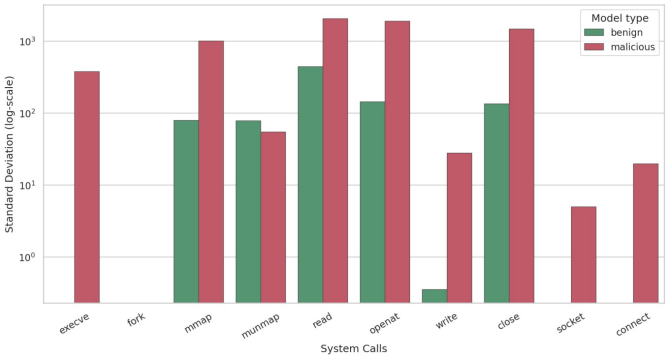


Fig. 1: Syscall comparison between DYNAHUG's benign training set (2000) vs real malicious models (25) in the *text-generation* cluster.

II. OVERVIEW

A. Problem Definition

In this work, we pose the following question: *Given a PTM, how can we automatically detect that it is malicious (or benign)?* Addressing this question is important to ensure the security of trusted execution environments and Model Hubs. Specifically, we aim to develop an automated method that ensures that malicious PTMs are identified during model loading or when uploaded on Model Hubs. A PTM is considered *malicious* if it exhibits insecure or unsafe behaviors that are beyond the intended behaviors of typical PTMs. For instance, PTMs that copy user data, sends user data to an unknown IP address or perform remote code execution are considered to be malicious [33], [34]. In contrast, a benign PTM from the `text-generation` task cluster is only expected to accept some input and output text. Thus, a malicious model detector is effective, if it detects genuinely malicious models and does not misclassify benign models. Otherwise, it is ineffective.

B. Key Insight

This work proposes DYNAHUG, an automated technique for malicious PTM detection. The main idea of DYNAHUG is to employ dynamic program analysis and ML to learn the behavior of benign PTMs. The *key insight* of our approach is that, *for a specific ML task, malicious PTMs often exhibit behaviors that are unique, rare or different from the behaviors of benign models.* In particular, we posit that for a specific ML task, (a) benign PTMs exhibit common behaviors, e.g., they often use similar system calls (with similar frequency) and (b) malicious PTMs often exhibit unique, outlier behaviors that are rarely exhibited by benign PTMs. Hence, we hypothesize that automatically learning the behaviors of benign PTMs is applicable for detecting malicious PTMs. Malicious PTMs are known to perform certain operations (e.g., remote code execution) that are never or rarely performed by benign PTMs [8]. This is evident from the fact that malicious payloads require certain system calls for attack orchestration. For instance, malicious PTMs that perform remote code execution often employ system calls (such as `execve`) which are rarely used by benign models (see Figure 1) since PTMs do not typically execute arbitrary code or require shell access [23]. This insight is evident in the literature; previous works have shown that certain behaviors (e.g., imports or system calls) are security sensitive, unique to malicious models or identifiable

TABLE II: Motivating examples showing the performance of DYNAHUG and baselines in detecting malicious PTMs. Disassembled code/trace snippets in orange are benign. Code/trace snippets in red shows malicious payloads ✓ meaning the malicious PTM was detected by a tool. ✗ mean the model was classified as benign and ✓indicates a false positive.

	Benign	**MALHUG**	**PyPI**
Model Name	llm-stacking/G_learn_depth[28]	jossefharush/gpt2-rs [29]	Zolllll/dont_download_this[30]
Description	Benign model importing a Tensor class from torch, not part of the standard library [31]	Model found in MalHug [8] containing blacklisted library eval [32]	Model injected with a payload using library execute from PyPI [30]

Disassembled Code	`110: \x93 STACK_GLOBAL`
	`111: \x94 MEMOIZE (as 7)`
	`112: \x8c SHORT_BINUNICODE 'torch'`
	`119: \x94 MEMOIZE (as 8)`
	`120: \x8c SHORT_BINUNICODE 'Tensor'`
	`128: \x94 MEMOIZE (as 9)`
	`129: \x93 STACK_GLOBAL`
	`130: \x94 MEMOIZE (as 10)`
		`0: \x80 PROTO 2`
		`2: c GLOBAL '__builtin__ eval'`
		`20: q BINPUT 0`
		`22: X BINUNICODE`
		`'exec(\'\'\'\nimport`
		`urllib.request;`
		`exec(urllib.request.`
		`urlopen("https://`
		`pastebin.com/raw`
		`/sVvZph7V").read().`
		`decode())\n\'\'\') or dict()'`
		`158: q BINPUT 2`
		`160: R REDUCE`
			`0: \x80 PROTO 2`
			`2: c GLOBAL 'execute both'`
			`16: q BINPUT 0`
			`18: X BINUNICODE 'nc -e /bin/sh`
			`127.0.0.1 4444'`
			`51: q BINPUT 1`
			`53: \x85 TUPLE1`
			`54: q BINPUT 2`
			`56: R REDUCE`
			`57: q BINPUT 3`

Dynamic Traces	`104089 newfstatat(AT_FDCWD,`
	`"/usr/src/app/.venv/lib/`
	`python3.10/site-packages/`
	`torch/_tensor.py",`
	`{st_mode=S_IFREG|0644, st_size=74282,`
	`...}, 0)= 0`
	`...`
	`104089 socket(AF_INET6,`
	`SOCK_STREAM|SOCK_CLOEXEC,`
	`IPPROTO_IP) = 3`
	`104089 bind(3, {sa_family=AF_INET6,`
	`sin6_port=htons(0),`
	`sin6_flowinfo=htonl(0),`
	`inet_pton(AF_INET6, "::1",`
	`&sin6_addr), sin6_scope_id=0},`
	`28) = 0`
	`...`
		`...`
		`370730 socket(AF_INET, SOCK_DGRAM`
		`|SOCK_CLOEXEC|SOCK_NONBLOCK,`
		`IPPROTO_IP) = 4`
		`370730 setsockopt(4, SOL_IP,`
		`IP_RECVERR, [1], 4) = 0`
		`370730 connect(4, sa_family=AF_INET,`
		`sin_port=htons(53),`
		`sin_addr=inet_addr`
		`("169.254.169.254"), 16) = 0`
		`370730 poll([{fd=4, events=POLLOUT}],`
		`1, 0) = 1 ([{fd=4,`
		`revents=POLLOUT}])`
		`...`
			`...`
			`410421 close(7) = 0`
			`410421 close(9) = 0`
			`410421 getdents64(5, 0x7ffcf8560c80 /*`
			`0 entries */, 280) = 0`
			`410421 close(5) = 0`
			`410421 execve("/bin/sh", ["/bin/sh",`
			`"-c", "nc -e /bin/sh 127.0.0.1`
			`4444"], 0x611b65855370 /* 46`
			`vars */ <unfinished ...>`
			`410416 <... vfork resumed>)`
			`= 410421`
			`...`

Fickling[24]	✓ (Tagged as unsafe)	✓	✓ (Tagged as unsafe)
PickleScan [20]	✗	✓	✗
ModelScan [21]	✗	✓	✗
HF_JFrog [19]	✗	✓	✗
HF_Guardian [17]	✗	✓	✗
HF_ClamAV [18]	✗	✗	✗
HF_PickleScan [18]	✓(marked as needing attention)	✓	✓(marked as needing attention)
ModelTracer [23]	✓	✓	✓
DYNAHUG	✗	✓	✓

as safe, unsafe or sensitive [8], [23].

C. Motivating Examples

Table II shows examples of real-world benign and malicious PTMs. The two malicious models (last two columns) allow an attacker to conduct dynamic execution of arbitrary Python code during model loading or deserialization. As shown in Table II ("MALHUG" third column), most tools detect the malicious payloads involving builtin Python library imports (e.g., `os`). In particular, such imports are often used to orchestrate dynamic code execution in Python. Thus, they are typically added to the blacklist rules of most detectors.

However, most tools are unable to detect uncommon or rare PyPI modules that allow for dynamic code execution ("PyPI" last column). State-of-the-art detectors miss such modules due to the non-exhaustive nature of blacklists. Indeed, there are numerous PyPI libraries that allow for dynamic code execution which makes it impossible to construct a complete blacklist.[2] Additionally, we show that some detectors (e.g., Fickling) flag almost all models as malicious. This is due to its non-exhaustive set of safe and unsafe opcodes and library imports. Overall, SOTA detectors suffer from under- or over- approximation due to static, incomplete sets of blacklisting or whitelisting rules.

Unlike most detectors, DYNAHUG detects the two malicious models in Table II and it correctly classifies the benign model. This is due to its use of *task-specific clustering, dynamic analysis* and *ML*, which allows it to accurately learn the behavior of benign models and flag the behaviors of malicious models, In DYNAHUG, dynamic analysis allows to collect behaviors of benign models, and ML allows to generalize learned behaviors beyond a static rule set. DYNAHUG is able

[2]The first 20 pages of PyPI when searching "execute" [35] contains over 20 modules that can perform reverse shell execution, many of which are not blacklisted by the state-of-the-art detectors (as at 10 Oct 2025).

to detect malicious PTMs using rare library imports (Table II column 1), since such imports use specific system calls which are rarely used by benign models. This demonstrates the importance of dynamic analysis. Our task-specific clustering allows to narrow down the set of behaviors that are unique to a specific category of PTMs. This is inspired by previous works which have demonstrated that specific categories of applications exhibit similar program behaviors [36], [37], [38].

Note on Traditional malware scanners: Traditional malicious app detectors are not amenable to malicious Pickle/PTM detectors due to the difference in the behavioural profile of PTMs vs. apps or traditional software. For instance, Hugging Face has two main types of traditional malware detectors (ClamAV and VirusTotal), and three specialised PTM scanners. At the time of our Hugging Face analysis, only the specialised scanners detect the malicious PTM datasets collected in this work and reported in previous works (MALHUG). In particular, the three specialised PTM scanners such as PickleScan and JFrog are more effective at detecting malicious PTMs than traditional scanners (see Table II). Furthermore, we uploaded five (5) models from MALHUG to VirusTotal [39] to demonstrate the issue. We note that none of the five (5) malicious models are detected [40], [41], [42], [43], [44] by any of the 64 security scanners on VirusTotal. These 64 scanners are from well-known security vendors like Kaspersky, SentinalOne, BitDefender and CrowdStrike. However, this same pickle file is easily detected by pickle security scanning tools like Picklescan [20].

D. Novelty vs. State-of-the-art

Table I illustrates the novelty of our approach (DYNAHUG) with respect to state-of-the-art techniques. Detectors can be divided into two classes, namely static detectors and dynamic detectors. On one hand, static detectors employ static analysis (e.g., disassembling, dataflow analysis or tainting) alongside blacklisting or whitelisting of specific imports, system calls or opcodes. For instance, PickleScan [20] employs a combination of static analysis, blacklisting and whitelisting of safe and unsafe imports and opcodes for malicious PTM detection (*see* Table I). On the other hand, there are few dynamic detectors. Notably, ModelTracer [23] combines dynamic analysis and a blacklist of system calls to detect malicious PTM. However, ModelTracer's blacklisting limits it from capturing rare or previously unseen imports or system calls and does not allow for subtlety in blacklisted calls. As exemplified in Table II (column two), ModelTracer detects the benign model as malicious because of the presence of a `socket` call, which is part of the blacklist. Additionally, traditional anti-virus (e.g. VirusTotal, ClamAV) are ineffective in detecting malicious PTMs since they are general software vulnerability scanners, they are not specialised for detecting anomalous behaviors in PTMs. This is evident in the poor performance of ClamAV on our motivating examples (Table II HF_ClamAV). In particular, anti-virus perform poorly on malicious PTMs due to the difference in the behavioral profile of PTMs versus traditional software. Meanwhile, DYNAHUG is able to correctly classify models as benign since it does not rely on a rulelist. The system-level granularity of DYNAHUG's traces contributes to its performance since the hundreds of imports/opcodes that are manually blacklisted/whitelisted by existing approaches translate to a limited set of system calls in DYNAHUG.

To the best of our knowledge, DYNAHUG is the first approach that employs ML and dynamic analysis for malicious PTM detection. DYNAHUG uniquely combines task-specific clustering, dynamic analysis and machine learning. Table I shows DYNAHUG is the *only* open-source tool that does not rely on blacklisting or whitelisting rules to detect malicious PTMs and the *only* detector that deploys *ML and dynamic analysis* for malicious PTM detection. ModelTracer, DYNAHUG is the only other tool that employs dynamic analysis. We also compare DYNAHUG to ModelTracer in section V (RQ2) and Table VI.

Recently, SafePickle [27] proposed an ML-based approach that uses *static* analysis of pickle files to detect malicious models. The best performing model, in their work, is a *supervised*-learning-based `RandomForest` detector, which was evaluated on their self-curated dataset. Due to proprietary reasons, the code to implement SafePickle and the resulting models are not available. In addition, its training data split are not detailed to support reproduction or comparison.

Unlike SafePickle, DYNAHUG is an ML-based approach that utilises dynamic features to learn and identify malicious behaviors in pickle models. DYNAHUG's learning approach is unsupervised to enable generalizability due to the imbalance of malicious vs. benign models available in the real world.

E. Threat Model

Attack assumptions: We assume that the attacker designs the attack to occur during PTM deserialization. In this work, we assume the attacker modifies an existing model or acts as a third-party model provider supplying a PTM containing arbitrary malicious payloads. The malicious model is distributed through Model Hubs, provided online or directly sent to users. The attacker provides the malicious model as well as instructions to load/execute the model. This setting is practical and fits how models are provided on Model Hubs (HF [1] and GitHub [3]).

Defense assumptions: We assume the defender loads the model following the instructions provided by the model provider. We do not require access to the source code of the model or need security expertise (e.g., to modify how the model is loaded), except the instructions provided by the provider (e.g., in the model card or README). We do not assume access to module libraries or need to modify the default model loading instruction. We assume that the PTM is executable in a sandbox. These assumptions are practical, relying on the trust of the victim in the Model Hubs and third-party vendors and does not require additional security or ML expertise or external tool except the model and the model loading instructions. This setting is realistic and similar to the manner end-users obtain PTMs from Model Hubs – HF [1], GitHub [3], Kaggle [2].

III. METHODOLOGY

Figure 2 illustrates the high-level workflow of DYNAHUG. Algorithm 1 (Appendix) describes the algorithm for DYNAHUG as detailed in subsection XI-A.

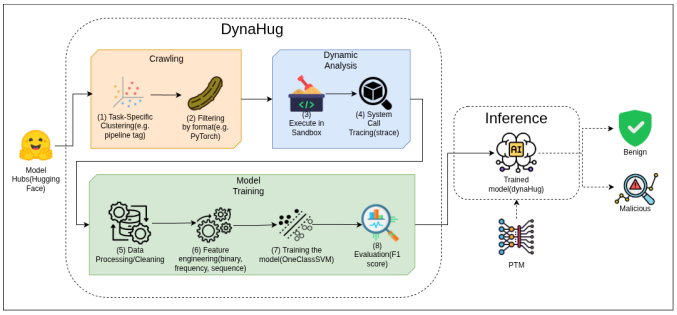


Fig. 2: Workflow of DYNAHUG

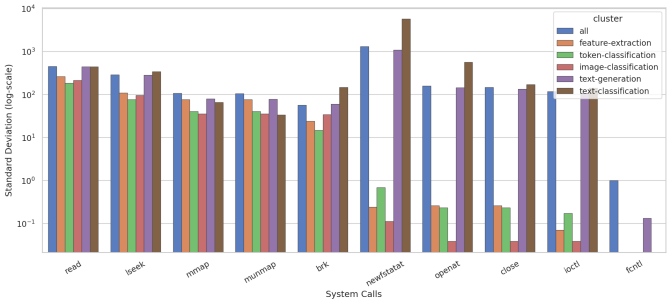


Fig. 3: Inter-cluster and Intra-cluster analysis of the system calls showing the standard deviation of the maximum difference in mean frequency between clusters.

A. Detailed Methodology

Crawling and Clustering: The goal of this phase is to collect relevant models from a Model Hub. We make use of the proprietary Model Hub API (e.g., `huggingface_hub [45]`) to filter the repositories based on the tag and the type of the model artifact as shown in Algorithm 1. Prior works, such as *Gorla et al.* [36], demonstrate that Android applications that are similar, in terms of their descriptions, should also behave similarly. Analogously, our underlying assumption is that models which are assigned the same tags should behave similarly. Task tags in Model Hubs identify the task a particular model is designed for. For instance, the `text-generation` pipeline tag in HF identifies models that are designed for text generation tasks. This helps narrow down the expected behaviour of a benign model from that particular tag. We also evaluate our task-clustering hypothesis by comparing the standard deviation (SD) of the mean frequency of the top-10 most differing system calls across clusters, using the top-2000 most liked models from HF. In particular, we examine the SD of the mean frequency of system calls for five (5) clusters, namely (a) within clusters (*intra-cluster*) for *text-generation*, *text-classification*, *feature-extraction*, *token-classification* and *image-classification*, and (b) without clustering (*inter-cluster,* "*all*"). Figure 3 shows differences in the SD of the mean frequency within and outside clusters. Thus, clustering captures the fine-grained behavior of the benign PTMs and ease malicious PTM detection.

In our analysis of HF, we rank the models according to the likes, and filter models based on pipeline tags with the help of the `huggingface_hub [45]` library in Python. The intuition is that popular models, models which have a higher amount of "likes", are less likely to be malicious. We focus our efforts on PyTorch models since 95% of all malicious models in HF are known to be PyTorch models [46]. Hence, when retrieving the models, we make sure that a `pytorch_model.bin` is present. We look for this file in particular since it is common convention within Hugging Face (HF) to store the model weights for inference inside `pytorch_model.bin`. This is to ensure that the `transformers` library from HF is able to find the weights file when trying to load a model for inference [47]. Next, the fetched models are stored in a Google Cloud Storage (GCS) Bucket [48] and sent to a sandbox for dynamic analysis.

1) Dynamic Analysis: DYNAHUG simulates an inference workflow of a user trying to run a model downloaded from a Model Hub. We build a `Docker` [49] container to load/deserialize the PyTorch models, emulating a reproducible environment similar to that of a user. `Docker` containers provide OS-level virtualization to isolate any processes running within it from the host machine, preventing any damage from potentially malicious models, e.g., Remote Code Execution (RCE) on the host machine [50]. Moreover, since the output logs from strace are non-deterministic and varies with the environment on which the PyTorch model is deserialized and the hardware resources available at that point in execution, it is imperative to have control on the environment, which `Docker` provides [51]. DYNAHUG provides the flexibility of choosing the runtime observability tools (e.g., `strace` [52], `tcpdump` [53], `eBPF` [54], etc.) to the user depending on the runtime aspects they wish to monitor. Once the runtime observability tool records the deserialisation process of the PyTorch model, the logs are saved for further processing in the upcoming phases.

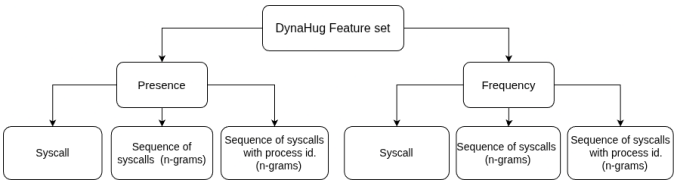
In our analysis, we utilize `strace`, a diagnostic userspace utility used to monitor interactions between processes and the Linux kernel which includes system calls, signal deliveries and change of process state [52]. `strace` monitors system calls related to File System Management, Networking, Process Management, Memory Management, etc. which covers almost all key operations to give a broad overview of system activity. Previous works [23] also utilize `strace` for dynamic analysis and detection of malicious PTMs. `strace` outputs a detailed raw log file which contains the system calls and their arguments. For instance, Example 1 shows the system call, `execve`, running the unix tool, `cat`, to display AWS secrets on the terminal output in a folder commonly known to store them.

2) Model Training: The following explains the model training process of DYNAHUG.

Data Processing: DYNAHUG extracts system call frequency and stores it in structured formats (e.g., CSV, Apache Parquet).

Feature Engineering: DYNAHUG prepares a presence and frequency feature set for system calls as shown in Figure 4 to feed into the classifier. Presence features indicate the presence of a certain feature with a binary value of either 0 or 1. Frequency features indicate the frequency of occurrence of a certain feature. These features provide insight on the

Fig. 4: DynaHug's feature set



Example 1: Strace log snippet from deserialization [33]

```
22928 execve("/usr/bin/cat", ["cat", "/home/sandbox/.aws/secrets"], 0x5d0f8b34df28 /* 55 vars */
      <unfinished ...>
22927 <... vfork resumed>)  = 22928
22927 rt_sigprocmask(SIG_SETMASK, [], ~[KILL STOP RTMIN RT_1], 8) = 0
22928 <... execve resumed>)  = 0
```

TABLE III: PTMs on Top six (6) Model Hubs.

Hub	#Models	Formats	Text Generation	Text Classification	Feature Extraction	Token Classification	Image Classification
Hugging Face [1]	1881296	Pytorch, Tflow, gguf	296411	98414	19683	30083	28347
GitHub [3]	150685	Pytorch, Tflow, gguf	1732	4427	1957	86	12314
Spark NLP [5]	135372	Custom	25181	35248	N/A	N/A	4336
OpenCSG [4]	111189	Pytorch, Tflow, gguf	4224	278	484	N/A	1006
ModelScope [6]	80574	Pytorch, Tflow, gguf	20307	1172	740	17	N/A
Kaggle [2]	3140	Pytorch, Tflow, gguf	800	197	N/A	1	72

statistical distribution of system calls during the deserialization process. To avoid magnitude bias towards certain system calls DynaHug normalizes frequency features to the same scale.

In our analysis, we utilize the `-c` flag in `strace` to get the summarized information of system calls (Section III-A1) during deserialization. The frequency of system calls is parsed from this summarized data and stored in Apache Parquet, since it is faster to load than CSV [55]. DynaHug fetches the presence and frequency features for plain system calls and applies normalization with the help of `StandardScaler(nomean=True)` class in `scikit-learn` [56]. Next, the established feature set is passed to the training component. We have also evaluated other features, e.g., n-grams and Process IDs (*see* section IV).

Training: DynaHug performs hyperparameter tuning while training a one-class SVM. The models trained from each parameter setting are evaluated using F1-score. The best performing model is chosen as DynaHug and used during inference. At inference, DynaHug follows a similar approach to training to retrieve and process the data from the model analysis (Section XI-A). The processed data is passed into DynaHug for detection – "Benign" or "Malicious".

IV. Experimental Setup

A. Research Questions

- **RQ1 Effectiveness:** How effective is DynaHug in detecting malicious PTMs?
- **RQ2 State-of-the-art Comparison:** How does DynaHug compare to the SOTA detectors?
- **RQ3 Ablation study:** What are the contributions of our design decisions to the performance of DynaHug?

We provide a detailed explanation of our research protocol in subsection XI-I (appendix).

B. Datasets

In our experiments, we employ Hugging Face (HF) as our Model Hub, since it is the most popular model hub with the largest set of models. Table III provides details of the top six model hubs. We collect our dataset of benign models from the Top-K (default 2000) models on HF. Our experiments involve top five (5) largest task categories from HF.

Table IV describes our collected dataset for each cluster. For the text-generation cluster, we first filter out models that are flagged by HF as malicious and that are not part of known malicious PTMs [8], [25]. We download the top 3000 models ranked by most likes, from which we filtered out two (2) models that are flagged by HF and MalHug [8] as malicious. We download another 1000 benign models so that we can evaluate DynaHug on them, and also inject them with malicious payloads to make our injected set of 2000 models. To leave an appropriate gap between the training set and the injection set to avoid contamination, we collect our next set of 1000 models from the 3600 to 4600 top liked models. Furthermore, we download the next 1025 models that are not flagged by HF as an evaluation set to test the model against, and balance our dataset with regards to our malicious set. The above setting describes all text-generation experiments (**RQ1** to **RQ4**). Then, we repeat the steps described above for the rest of the task clusters (**RQ1**) and non-clustered experiments (**RQ4** (Appendix)).

C. Injected malicious models

To ensure detectors generalise to unseen models and address class imbalance, we augment our evaluation dataset by injecting malicious payloads into benign models using models from HF and malicious payloads from MalHug and libraries from PyPI that allow for code execution.

MalHug injected models: Using Fickling [24], we automatically inject 64 malicious payloads from MalHug into 1000 benign models from HF. Example 2 (appendix) shows an example of a MalHug injected payload. We ensure a model is valid by checking that it is successfully loaded by PyTorch with `torch.load()`. Injection process times out and proceeds to the next payload when there is a blocking operation from a MalHug payload, e.g., it requires user input (`input()`).

PyPI injected models: The goal of the PyPI injected model is to provide more realistic malicious payloads (e.g., used for information reconnaissance, or reverse shells), since most MalHug payloads are Proof-Of-Concepts (PoCs). Hence, the payloads were chosen from well-known external sources such as PayloadsAllTheThings [57] and revshells.com [58]. We automatically inject 20 of these malicious payloads using PyPI libraries [59] into 1000 benign models from HF. Example 3 (appendix) shows an example of our PyPI injected payload. These libraries allows the user to either execute system commands (similar to `os.system()`), or execute Python code (similar to `exec()`). Using these 20 malicious pickles, we directly inject their Pickle bytecode into the 1000 benign models we collected, after modifying the required stack addresses such that there is no memory conflict between the original model and the injected payload. For each injected payload, we setup a test oracle to confirm that the resulting models are valid, and injection was successful. To account for the blocking operations

TABLE IV: Details of training and evaluation datasets across all clusters.

Datasets	Range	Benign						Malicious						Total
	(Sorted By Most Likes)	Text Gen	Text Class	Feature Extraction	Token Class	Image Class	No Cluster	Text Gen	Text Class	Feature Extraction	Token Class	Image Class	No Cluster	
Real (HF)	$(1-3000) \cup (3601-5625)$	5025	5001	5001	5000	5000	3000	3	0	0	0	0	0	28030
Real (MALHUG)	-	0	0	0	0	0	0	20	2	17	0	0	0	39
Real (PickleBall)	-	0	3	16	0	0	0	2	2	0	0	0	0	23
Synthetic (MALHUG Injected)	3600-4600	0	0	0	0	0	0	1000	1000	1000	1000	1000	1000	6000
Synthetic (PyPI Injected)	3600-4600	0	0	0	0	0	0	1000	1000	1000	1000	1000	0	5000
Real (Total)	-	5025	5004	5017	5000	5000	3000	25	4	17	0	0	0	28092
Synthetic (Total)	-	0	0	0	0	0	0	2000	2000	2000	2000	2000	1000	11016

of the reverse shells, we start a netcat [60] server that sends an `exit` instruction to terminate the reverse shell.

D. Training Process

Architecture Selection: The one-class SVM (OCSVM) was selected since it is a popular choice in unsupervised learning based anomaly detection, alongside Stochastic Gradient Descent one-class SVM (SGDSVM) and Isolation Forest (IF). These architectures require only one-class of training data, which are necessary in the PTM setting since there are millions of benign models but few malicious models.

Feature Engineering: DYNAHUG has three (3) different features sets for training (see Figure 4). First, *presence and frequency* features capture the presence and frequency of individual system calls (e.g.,"execve", "read", "write"). For the presence features, an exhaustive list of syscalls were taken from the Linux kernel source code in the docker container [61], corresponding to the x86-64 architecture to prevent Out of Vocabulary errors. *Sequence* features represent the presence and frequency of system call sequences (n-grams) (e.g., "execve:read", "read:write"). *Sequence* features are represented with 2-grams, as this configuration offers the best trade off between model complexity and performance as shown in Table X (appendix). Although higher order n-grams (i.e., 3-gram, 4-gram) provide a better spacial understanding of system calls to the model, preliminary analysis showed that choosing these n-gram configurations would lead to poorer performance. Finally, *Process Sequence* based features incorporated presence and frequency of process ID (PID) system call sequences (e.g., "P1:close_P5:rt_sigprocmask", "P4:rseq_P1:read"). The raw PID value was not used to represent the process since each process has a unique PID in every run which would introduce noise into our training dataset. Hence, a generalized form of the PID is used where the parent process is P1 and any subsequent processes would be numbered accordingly. Process sequence features are also 2-grams.

Dataset Split: Datasets are split into train, validation and test sets in the ratio 80:10:10. The training dataset consists exclusively of the benign models collected from HF. However, for the validation and test set, we combined an equal number of benign and malicious models. Specifically, the malicious models were randomly sampled from MALHUG and MALHUG-injected from the chosen task cluster. This makes it easier to facilitate a more reliable assessment on the model's ability to distinguish between benign and malicious models. The malicious models in the validation or test set contains half of MALHUG from the selected task cluster, and the rest are supplemented by MALHUG-injected to solve the dataset imbalance between benign and malicious models in the validation and test set. Data processing is described in subsection XI-F (appendix).

Hyperparameter Tuning Strategy: DYNAHUG performs Grid Search on a set of hyperparameters which are tailored to a specific classifier architecture. We employ a 5-fold cross validation procedure to ensure that we get the average F1-score for the best model evaluation. Hyperparameter choices and the metrics used are described in the subsection XI-G (appendix) and subsection XI-H (appendix).

E. Implementation Details and Platform

DYNAHUG was implemented in about 4.1k lines of Python code. Experiments and data analysis were implemented in 6k lines of Python code. Experiments were setup on n2-highmem-4 (4 vCPUs, 32 GB Memory) Google Cloud Compute Engine instance with Ubuntu, 22.04 LTS Minimal installed. Within this VM, a docker container with a base image of `python:3.10.12-slim` was setup with all project files cloned from the GitHub repository and the required dependencies installed. The LLM baseline experiment was run on Google Colab [62] with a NVIDIA-A100 GPU [63]. HF `transformers` library was utilized to run inference on Llama-3.1. GPT-5.2 was executed through the official OpenAI API.

V. RESULTS

RQ1: Effectiveness. We found that DYNAHUG *is effective in detecting malicious PTMs across all five (5) clusters*. Table V reports our results. Across all metrics, DYNAHUG's performance is consistently high (between 0.9749 and 0.998). DYNAHUG performs well on both real malicious models and injected malicious models. On the one hand, DYNAHUG has a True Positive Rate (TPR) of 100%, 100% and 76.47% on all real malicious models for `text-generation`, `text-classification` and `feature-extraction` clusters, respectively. On the other hand, it has a TPR of 99.3%, 99.25% and 99.7% on the synthetic injected models for `text-generation`, `text-classification` and `feature-extraction` clusters, respectively. It detects 100% of injected malicious models for `token-classification` and `image-classification`.

Overall, DYNAHUG has a good performance (>=0.975) across all metrics and settings. This result implies that the ML-based dynamic analysis technique of DYNAHUG is effective in malicious PTM detection. In summary, our approach (DYNAHUG) generalizes to five different PTM tasks.

TABLE V: DYNAHUG's performance on five (5) task-based clusters. Here x/y implies that x is detected out of y. **"TPR"** and **"FPR"** mean **"True Positive Rate"** and **"False Positive Rate"** respectively.

Cluster	Benign (FPR) Real	Malicious (TPR) Real	Malicious (TPR) Injected	Real Precision	Real Recall	Real F1-score	Injected (2000) Precision	Injected (2000) Recall	Injected (2000) F1-score	Total Precision	Total Recall	Total F1-score
text-generation	1/2025 (0.04%)	25/25 (100%)	1986/2000 (99.30%)	1	1	1	0.9995	0.993	0.9962	0.9995	0.9931	0.9963
text-classification	28/2004 (1.39%)	4/4 (100%)	1985/2000 (99.25%)	1	1	1	0.9861	0.9925	0.9893	0.9861	0.9925	0.9893
feature-extraction	18/2017 (0.89%)	13/17 (76.47%)	1994/2000 (99.70%)	1	0.7647	0.8667	0.9911	0.997	0.994	0.9911	0.9950	0.9930
token-classification	103/2000 (5.15%)	N/A	2000/2000 (100%)	N/A	N/A	N/A	0.951	1	0.9749	0.951	1	0.9749
image-classification	8/2000 (0.4%)	N/A	2000/2000 (100%)	N/A	N/A	N/A	0.996	1	0.998	0.996	1	0.998

TABLE VI: Comparison of DYNAHUG vs. open-source SOTA's performance.

Analysis Type	Detector	Benign Real (2025)	Malicious (2025) Real	Malicious (2025) MALHUG	Malicious (2025) PyPI	Overall Performance TP	TN	FP	FN	Precision	Recall	F1-score
Static	PickleScan [20]	**0**	23	**1000**	44	1067	**2025**	**0**	958	**1**	0.5269	0.6902
	ModelScan [21]	**0**	23	**1000**	44	1067	**2025**	**0**	958	**1**	0.5269	0.6902
	Fickling [24]	2025	**25**	**1000**	**1000**	**2025**	0	2025	**0**	0.5	**1**	0.6667
Dynamic	ModelTracer [23]	**0**	**25**	828	907	1760	**2025**	**0**	265	**1**	0.8691	0.9299
Dynamic + LLM	Llama-3.1 [64]	1337	22	993	995	2010	688	1337	15	0.6005	0.9926	0.7483
	GPT-5.2 [65]	12	20	988	796	1804	2013	12	221	0.9933	0.8909	0.9393
Dynamic	DYNAHUG (default)	1	**25**	**1000**	986	2011	2024	1	14	0.9995	0.9931	**0.9963**

DYNAHUG *is effective in detecting malicious PTMs across all five (5) clusters (up to 0.998 F1-score).*

RQ2: State-of-the-art Comparison. We found that DYNAHUG *is up to 44% more effective than the state-of-the-art detectors in terms of F1-score.* It *outperforms the other baselines by up to 25%, 20.77% and 19.2% in detecting Real Malicious,* MALHUG *injected and PyPI injected respectively.* Table VI shows that DYNAHUG outperforms the closest competitior (GPT-5.2) by about 5% (0.9963 vs. 0.9393 F1-score). GPT-5.2 excels in detecting the MALHUG injected set as compared to Llama-3.1 and ModelTracer. ModelTracer is the best non-LLM detection in the baselines. On the one hand, ModelTracer detects all real malicious PTMs, but it does not generalize on the exact malicious payloads when injected in different models. This demonstrates that its detection is not generalizable to new models. In addition, ModelTracer does not perform well on PyPI injected modules, which underscores its non-exhaustive trace blacklisting. On the other hand, the static detectors have the worst performance, followed by the Llama-3.1-8B-Instruct-tuned. DYNAHUG is 44% (0.9963 vs. 0.6902 F1-score) more effective than the best static detectors, i.e., PickleScan and ModelScan. We attribute the poor performance of the static detectors to the lack of behavioral information, use of rule sets and the over-approximation of static analysis. Finally, DYNAHUG is 33% (0.9963 vs. 0.7483) more effective than Llama-3.1 in malicious PTM detection. The poor performance of Llama-3.1 is attributed to its lack of PTM-specific security knowledge. This experiment demonstrates the superiority of DYNAHUG, and the importance of its behavioral analysis and learning approach to malicious PTM detection.

DYNAHUG *is up to 44% more effective than the state-of-the-art detectors in terms of F1-score.*

RQ3: Ablation and Probing study (Analysis Type). We found that DYNAHUG *with dynamic analysis is more effective than* DYNAHUG *with static analysis.* Table VII shows that dynamic analysis in DYNAHUG is twice (2x) as effective as

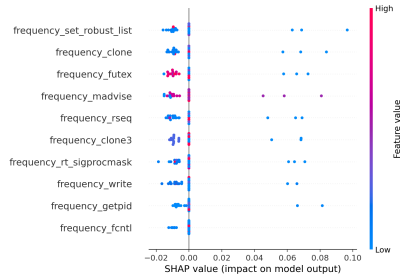


Fig. 5: Top-10 most important features that explain DYNAHUG's predictions on malicious models

static analysis (precision of 0.5 vs. 0.9995). Meanwhile, hybrid analysis has the same performance as dynamic analysis. We observe that static analysis is not as effective as dynamic analysis. Besides, combining dynamic and static analyses does not improve the performance of DYNAHUG, it is as effective as using *only* dynamic analysis (*see* Table VII). We attribute the weak performance of static analysis to the lack of precise execution information which causes over-approximation. This result also aligns with the performance of the static detectors in **RQ2** (*see* Table VI) and shows that DYNAHUG's dynamic analysis contributes positively to its detection effectiveness.

Dynamic analysis contributes positively to the effectiveness of DYNAHUG*; it is twice as effective as static analysis.*

Feature Selection: Results show that *the default feature setting of* DYNAHUG *(presence and frequency of system calls) contributes positively to its effectiveness and outperforms alternative features setting.* More importantly, Table VIII shows that the *frequency of system calls* improves DYNAHUG's detection effectiveness more than all other feature settings. In comparison to presence only features, the frequency of system calls improves the recall and F1-score of DYNAHUG by up to 19% (0.8326 vs. 0.9931) and ≈10% (0.9086 vs. 0.9963), respectively. Meanwhile, employing a larger feature set than the default settings (e.g., adding sequence of system calls and/or sequence of system processes) leads to poorer performance

TABLE VII: DYNAHUG's performance under different analysis types (best results are in **bold** text)

Analysis Type	Detector	Benign	Malicious		Overall Performance						
		HF (2025)	Real (25)	Injected (2000)	TP	TN	FP	FN	Precision	Recall	F1-score
Static	DYNAHUG	2025	**25**	**2000**	**2025**	0	2025	**0**	0.5000	**1.0000**	0.6667
Dynamic	DYNAHUG (default)	**1**	**25**	1986	2011	**2024**	**1**	14	**0.9995**	0.9931	**0.9963**
Hybrid	DYNAHUG	**1**	**25**	1986	2011	**2024**	**1**	14	**0.9995**	0.9931	**0.9963**

TABLE VIII: DYNAHUG's effectiveness with varying feature sets showing 'freq" = frequency, "seq" = sequence and "proc" = process (best results are in **bold** text)

Feature Set	Detector	Benign	Malicious		Overall Performance						
		HF (2025)	Real (25)	Injected (2000)	TP	TN	FP	FN	Precision	Recall	F1-score
presence	DYNAHUG	0	22	1664	1686	**2025**	0	339	**1.0000**	0.8326	0.9086
freq	DYNAHUG	**1**	**25**	1986	**2011**	**2024**	**1**	14	0.9995	0.9931	**0.9963**
presence, freq	DYNAHUG (default)	**1**	**25**	1986	**2011**	**2024**	**1**	14	0.9995	0.9931	**0.9963**
presence, freq, seq	DYNAHUG	33	24	**2000**	**2024**	1992	33	**1**	0.9840	**0.9995**	0.9917
presence, freq, seq, proc seq	DYNAHUG	51	24	**2000**	**2024**	1974	51	**1**	0.9754	**0.9995**	0.9873

TABLE IX: DYNAHUG's performance across different architectures (best results are in **bold** text)

Architecture	Detector	Benign	Malicious		Overall Performance						
		HF (2025)	Real (25)	Injected (2000)	TP	TN	FP	FN	Precision	Recall	F1-score
IsolationForest	DYNAHUG	4	24	1782	1806	2021	4	219	0.9978	0.8919	0.9419
OneClassSVM	DYNAHUG (default)	**1**	**25**	1986	**2011**	**2024**	**1**	14	**0.9995**	0.9931	**0.9963**
SGDOneClassSVM	DYNAHUG	67	**25**	**2000**	**2025**	1958	67	**0**	0.9680	**1.000**	0.9837

with a reduced the precision and F1-score (*see* Table VIII). We attribute the performance of DYNAHUG's *default* setting to its use of the *frequency of system calls* which makes DYNAHUG detect outlier system call distribution. This observation aligns with our explainability findings. For instance, Figure 5 shows that the frequency of `set_robust_list, clone and futex` calls are the most important features for predicting malicious models. Similarly, the frequency of `futex, brk` and `read` are the most important features for predicting benign models as shown in Figure 6 (Appendix). This shows that DYNAHUG's feature setting contributes to its detection effectiveness.

The default feature setting of DYNAHUG *outperforms alternative feature settings and the frequency of system calls contributes the most to its performance.*

ML Architecture: We observed that *the default model architecture in* DYNAHUG *(*`OCSVM`*) outperforms the tested alternative architectures (*`IF` *and* `SGDOCSVM`*).* Table IX shows that default DYNAHUG (OCSVM) performs best in terms of precision and F1-score: DYNAHUG (`OCSVM`) performs best (F1 = 0.9963), followed by DYNAHUG with `SGDOCSVM` (F1 = 0.9419), then DYNAHUG with `IF` (F1 = 0.9837). However, we observed that DYNAHUG with `SGDOCSVM` has a slightly better recall (1.0 vs. 0.9931). We attribute the better performance of the the default DYNAHUG (`OCSVM`) to the size of our dataset, since the tested alternatives (`IF` and `SGDOCSVM`) are often sensitive to the size of the training dataset and more suitable for training larger datasets [66], [67], [68]. `OCSVM` is more suitable for DYNAHUG due to DYNAHUG's task clustering step (causing a reduced training size) and the huge computational cost of training larger datasets. This demonstrates that `OCSVM` fits DYNAHUG's design and outperforms alternatives (`IF` or `SGDSVM`).

DYNAHUG*'s architecture (*`OCSVM`*) fits its design and outperforms alternatives (*`IF` *or* `SGDSVM`*).*

Additional Experiments: We conducted additional probing and sensitivity study of DYNAHUG (**Appendix-RQ4**). We found that task clustering positively contributes to DYNAHUG's performance. DYNAHUG's performance improves as the training set grows. We also examine DYNAHUG's scalability to advanced attacks (**Appendix-RQ5**). Results show that DYNAHUG scales to advanced attacks (e.g., Anti-VM, Anti-debugging, delayed execution, etc.). subsection XI-J (Appendix) provides more details on the probing, sensitivity and advanced payloads experiments.

VI. LIMITATIONS AND THREATS TO VALIDITY

Internal Validity: The main threats to internal validity of this work and DYNAHUG are the correctness of our implementation and the reproducibility of our results. To mitigate this threat, we have tested our implementation and resulting models under varying settings to ensure reproducibility (*see* **RQ3**). We also provide our code and data to support scrutiny and reuse.

External Validity: The main risk to external validity is the generalizability of DYNAHUG and our findings. In particular, our findings may not generalize to other Model Hubs, tasks or unpopular (non-Top-K) models. To mitigate this threat, we have performed our experiments using the most popular model hub (Hugging Face), tested on two popular clusters and thousands of benign and malicious models. We have also tested on different parameter settings in our ablation and sensitivity study (**RQ3** and **RQ4**). In addition, to address concerns that the injected payloads may not be representative of real world malware, we have conducted experiments to test our approach on advanced payloads (e.g., Anti-VM detection techniques, staged payloads, obfuscated payloads, and delayed execution payloads) as

detailed in subsection XI-J (Appendix). However, despite best efforts, we acknowledge that our findings may not generalize to other settings, beyond the tested settting. The dynamic features explored in this work are not exhaustive. Employing additional features which may improve the performance of DYNAHUG. In future work, we plan to explore additional features such as API call graph, control flow graph, etc. [69], [70].

Construct Validity: To avoid experimenter bias in this work, we have employed thousands of models from Hugging Face and related scanners (e.g., MALHUG). We have also compared our approach to five (5) open-source state-of-the-art detectors and filtered our training set using the scanners provided by Hugging Face security API. Finally, we have also manually inspected the real-world malicious PTMs to confirm they are malicious and constructed test oracles to check that our injector indeed inject malicious payloads.

VII. RELATED WORKS

Static Detectors: These tools aim to detect malicious PTMs by inspecting its code for potential malicious behaviors, e.g., using disassemblers like pickletools [25] or Fickling [25]. Detection is typically performed through a blacklist of unsafe opcodes (e.g., PickleScan, Modelscan, HF PickleScan [20], [21], [18]) or a whitelist of safe opcodes (e.g., PickleBall [23]). To further overcome the limits of a static list of rules, some static detectors also employ methods such as heuristics (MALHUG [8]), policy generation (PickleBall [25]) or data flow analysis (Fickling [24]). Unlike these approaches, DYNAHUG employs dynamic behavior of PTMs rather than static analysis and relies on ML rather than static blacklisting or whitelisting.

Dynamic Detectors: Similar to DYNAHUG, ModelTracer [23] detects malicious PTMs using dynamic analysis, in particular, strace [52] and Python's sys [71]. It processes the generated syscall data and then checks for sensitive syscall presence like `exec`, `connect` and `chmod` as indicators of malicious behaviour. Invariably, it employs a blacklist of system calls to detect malicious PTMs. Unlike ModelTracer, DYNAHUG relies on the ML classifier to learn the behaviours of benign models, rather than relying on a list of sensitive calls that are rigid, can be easily bypassed and require expert review for edge cases.

VIII. CONCLUSION

This work proposes a novel method for detecting malicious PTM models. The goal of our work is to ensure that PTM users can detect malicious PTM models provided by third-party vendors in the ML supply chain system. In particular, we aim to ensure the safety of PTMs uploaded on Model Hubs like Hugging Face or executed in trusted user environments. Our approach (DYNAHUG) employs a combination of dynamic analysis, task-specific clustering and ML to detect malicious PTM models. We orchestrate DYNAHUG by employing a one-class SVM to learn the behavior of the top-K (2000) most liked models in a task-specific cluster (e.g., text generation) of Hugging Face. We also evaluate DYNAHUG using over 39,000 benign and malicious models from different sources, including Hugging Face, MALHUG and injected malicious payloads using MALHUG and PyPI modules. Besides, we compare the effectiveness of DYNAHUG to five baseline detectors, including static, dynamic and LLM-based detectors. Our evaluation results demonstrate that DYNAHUG is effective in detecting malicious PTMs, outperforms the baselines. This work motivates the need to employ behavioral analysis for PTM detection and reduce reliance on blacklists or whitelists. In the future, we aim to study additional model hubs and deploy DYNAHUG in the wild.

IX. DATA AVAILABILITY

For the reproducibility and reuse of DYNAHUG, we provide its appendix, source code and our experimental data:

https://dynahug-detector.github.io/

The artefact provides the source code and raw and processed `strace` logs of the models used in the experiments. Due to the size of benign PTM files used in our experiments (over 28K benign models, totaling several terabytes), we do not release them publicly. Instead, we provide CSV files with the metadata of all training and evaluation models used to support reproducibility. For security reasons, we do not publicly provide all of our injected malicious PTMs or make them available on any platforms, except as an artefact for this paper's evaluation. We plan to provide the PTMs as an artefact for research purposes after paper acceptance. In the artefact, we will include cautions, disclosure and disclaimer. In particular, we will provide the injected models and a description of the associated libraries for research purposes.

X. ETHICAL CONSIDERATIONS

DYNAHUG is computationally less intensive than using LLMs because our approach relies on classical ML algorithms with 30+ parameters and requires smaller datasets (39K PTM traces), as compared to the billions or trillions of parameters and trillions of tokens of publicly available training datasets in LLMs [72], [73], [74]. Hence, our approach is more sustainable and environmentally friendly compared to inferencing, training or fine-tuning LLMs.

In our study, we tested five (5) malicious PTMs on Hugging Face as a Proof-of-concept (PoC) to report the performance of Hugging Face scanners on our PyPI-injected models. These models contain PoC payloads that only demonstrate the capability of the attack without actually carrying out a real attack. We created an HF repository with some PoCs, and explicitly caution against downloading them. The repository's [30] card README contains a warning that it is intended for security research purposes and should not be downloaded: "Please do not download these models, as they are for research purposes only and may be damaging to your system. Not safe."

REFERENCES

[1] "Hugging Face – The AI community building the future." https://huggingface.co/, 2025, [Accessed 07-11-2025].

[2] "Kaggle: Your Machine Learning and Data Science C ommunity," https://www.kaggle.com/, 2025, [Accessed 07-11-2025].

[3] "GitHub," https://github.com/, 2025, [Accessed 07-11-2025].
[4] "OpenCSG," https://opencsg.com/, 2025, [Accessed 07-11-2025].
[5] "Spark NLP," https://sparknlp.org/, 2025, [Accessed 07-11-2025].
[6] "ModelScope," https://modelscope.cn/home, 2025, [Accessed 07-11-2025].
[7] Ronik, 2024. [Online]. Available: https://weam.ai/blog/guide/huggingface-statistics/
[8] J. Zhao, S. Wang, Y. Zhao, X. Hou, K. Wang, P. Gao, Y. Zhang, C. Wei, and H. Wang, "Models are codes: Towards measuring malicious code poisoning attacks on pre-trained model hubs," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. ACM, Oct. 2024, p. 2087–2098. [Online]. Available: http://dx.doi.org/10.1145/3691620.3695271
[9] The Hacker News, "New hugging face vulnerability exposes ai models to supply chain attacks," https://thehackernews.com/2024/02/new-hugging-face-vulnerability-exposes.html, 2024, [Accessed 09-11-2025].
[10] H. News, "Over 100 malicious ai/ml models found on hugging face platform," https://thehackernews.com/2024/03/over-100-malicious-aiml-models-found-on.html, 2024, [Accessed 09-11-2025].
[11] T. H. News, "Malicious ml models on hugging face leverage broken pickle format to evade detection," https://thehackernews.com/2025/02/malicious-ml-models-found-on-hugging.html, 2025, [Accessed 09-11-2025].
[12] E. Montalbano, (https://www.darkreading.com/application-security/hugging-face-ai-platform-100-malicious-code-execution-models), Feb 2024.
[13] K. Poireault, "Malicious AI Models on Hugging Face Exploit Novel A ttack Technique," https://www.infosecurity-magazine.com/news/malicious-ai-models-hugging-face/, 2025, [Accessed 09-11-2025].
[14] "Malicious ML models discovered on Hugging Face platform | ReversingLabs," https://www.reversinglabs.com/blog/rl-identifies-malware-ml-model-hosted-on-hugging-face, 2025, [Accessed 09-11-2025].
[15] "Federal register :: Request access," https://www.federalregister.gov/documents/2023/11/01/2023-24283/safe-secure-and-trustworthy-development-and-use-of-artificial-intelligence, 2023, [Accessed 09-11-2025].
[16] "cve.org," https://www.cve.org/CVERecord/SearchResults?query=pickle, 2025, [Accessed 09-11-2025].
[17] H. Face, "Third-party scanner: Protect ai," https://huggingface.co/docs/hub/en/security-protectai, 2025, hugging Face documentation, accessed: 2025-10-11.
[18] HuggingFace, "Pickle scanning (hub documentation)," https://huggingface.co/docs/hub/en/security-pickle#what-we-have-now, 2025, hugging Face documentation, accessed: 2025-10-11.
[19] H. Face, "Third-party scanner: Jfrog," https://huggingface.co/docs/hub/en/security-jfrog, 2025, hugging Face documentation, accessed: 2025-10-11.
[20] mmaitre314, "picklescan: Security scanner detecting python pickle files performing suspicious actions," https://github.com/mmaitre314/picklescan, 2025, gitHub repository, accessed: 2025-10-11.
[21] P. AI, "Modelscan: Protection against model serialization attacks," https://github.com/protectai/modelscan, 2025, gitHub repository, accessed: 2025-10-11.
[22] "Execute – pypi.org," https://pypi.org/project/execute/, 2010, [Accessed 10-11-2025].
[23] B. Casey, J. C. S. Santos, and M. Mirakhorli, "A large-scale exploit instrumentation study of ai/ml supply chain attacks in hugging face models," 2024. [Online]. Available: https://arxiv.org/abs/2410.04490
[24] T. of Bits, "Fickling: A python pickling decompiler and static analyzer," https://github.com/trailofbits/fickling, 2025, gitHub repository, accessed: 2025-10-11.
[25] A. D. Kellas, N. Christou, W. Jiang, P. Li, L. Simon, Y. David, V. P. Kemerlis, J. C. Davis, and J. Yang, "Pickleball: Secure deserialization of pickle-based machine learning models (extended report)," 2025. [Online]. Available: https://arxiv.org/abs/2508.15987
[26] P. Contributors, "weights_only_unpickler.py – pytorch," https://github.com/pytorch/pytorch/blob/main/torch/_weights_only_unpickler.py, 2025, github repository, Accessed: 2025-11-10.
[27] H. Ohayon, D. Gilkarov, and R. Dubin, "Safepickle: Robust and generic ml detection of malicious pickle-based ml models," 2026. [Online]. Available: https://arxiv.org/abs/2602.19818
[28] "llm-stacking/G_learn_depth at main — huggingface.co," https://huggingface.co/llm-stacking/G_learn_depth/tree/main, 2024, [Accessed 11-11-2025].
[29] "jossefharush/gpt2-rs at main — huggingface.co," https://huggingface.co/jossefharush/gpt2-rs/tree/main, 2023, [Accessed 28-01-2026].
[30] Zollllll, ""dont_download_this" – model card," https://huggingface.co/Zollllll/dont_download_this, 2025, accessed: 2025-11-10.
[31] "pytorch/Phi-4-mini-instruct-FP8 at main — huggingface.co," https://huggingface.co/pytorch/Phi-4-mini-instruct-FP8/tree/main, 2025, [Accessed 10-11-2025].
[32] F. Hijazi, ""totally-harmless-model" – model card," https://huggingface.co/FarisHijazi/totally-harmless-model, 2025, accessed: 2025-11-10.
[33] "ankushvangari-org2/unsafe_model · Hugging Face," https://huggingface.co/ankushvangari-org2/unsafe_model, 2023, [Accessed 09-11-2025].
[34] "star23/round2," https://huggingface.co/star23/round2/tree/main, 2023, [Accessed 09-11-2025].
[35] "Execute Search — pypi.org," https://pypi.org/search/?q=execute, 2025, [Accessed 11-11-2025].
[36] A. Gorla, I. Tavecchia, F. Gross, and A. Zeller, "Checking app behavior against app descriptions," in *Proceedings of the 36th international conference on software engineering*, 2014, pp. 1025–1035.
[37] T. S. R. Pimenta, F. Ceschin, and A. Gregio, "Androidgyny: Reviewing clustering techniques for android malware family classification," *Digital Threats*, vol. 5, no. 1, Mar. 2024. [Online]. Available: https://doi.org/10.1145/3587471
[38] D. George, A. Mauldin, J. Mitchell, S. Mohammed, and R. Slater, "Static malware family clustering via structural and functional characteristics," *SMU Data Science Review*, vol. 7, no. 2, p. 4, 2023.
[39] "Virustotal - virustotal.com," https://www.virustotal.com/gui/home/upload, [Accessed 16-04-2026].
[40] "drhyrum/bert-tiny-torch-picklebomb-virustotal," https://www.virustotal.com/gui/file/c8354611d7d2f2437aa2ce1af05760ae2a4acbfc21e2b3a4cbe77024130bf8df, [Accessed 16-04-2026].
[41] "zpbrent/reuse - virustotal," https://www.virustotal.com/gui/file/2337248d2e27e905e483ce8d1740035a98fd915d01c575fb5235d60016b8a56a, [Accessed 16-04-2026].
[42] "Farishijazi/totally-harmless-model - virustotal," https://www.virustotal.com/gui/file/65a011d547d25bd600580861f8d47447b6fedc4fb11d4c01ed348ef58e510b3b, [Accessed 16-04-2026].
[43] "Narsil/totallysafe - virustotal," https://www.virustotal.com/gui/file/5949e26161da6e53cf2771aa7de4212b70d5aa065baab49e600bb295f11fad11, [Accessed 16-04-2026].
[44] "admko/evil_test - virustotal," https://www.virustotal.com/gui/file/e7191fdaf9bc9b19266e8bcf348788a4493cdbe03baadf67a483b7093783aa4b , [Accessed 16-04-2026].
[45] "Hub client library — huggingface.co," https://huggingface.co/docs/huggingface_hub/index, 2025, [Accessed 07-11-2025].
[46] D. Cohen, "Data scientists targeted by malicious hugging face ml models with silent backdoor," https://jfrog.com/blog/data-scientists-targeted-by-malicious-hugging-face-ml-models-with-silent-backdoor/, 2024, [Accessed 27-10-2025].
[47] "huggingface/transformers github," https://github.com/huggingface/transformers/blob/main/src/transformers/modeling_utils.py, 2025, [Accessed 13-10-2025].
[48] "Cloud Storage," https://cloud.google.com/storage?hl=en, 2025, [Accessed 11-11-2025].
[49] I. Docker, "Docker: Accelerated Container Application Development — docker.com," https://www.docker.com/, 2025, [Accessed 06-11-2025].
[50] "What is a Container? | Docker — docker.com," https://www.docker.com/resources/what-container/, 2025, [Accessed 11-11-2025].
[51] Canonical, "Strace man page." [Online]. Available: https://manpages.ubuntu.com/manpages/jammy/man1/strace
[52] "strace," https://github.com/strace/strace, 2025, [Accessed 12-10-2025].
[53] "Tcpdump; libpcap," https://www.tcpdump.org/, 2025, [Accessed 12-10-2025].
[54] "ebpf - introduction, tutorials & community resources — ebpf.io," https://ebpf.io/, 2025, [Accessed 12-10-2025].
[55] F. PACULL, "Loading data into a Pandas DataFrame – a performance study," https://www.architecture-performance.fr/ap_blog/loading-data-into-a-pandas-dataframe-a-performance-study/, 2019, [Accessed 14-10-2025].
[56] "User Guide — scikit-learn.org," https://scikit-learn.org/stable/user_guide.html, 2025, [Accessed 07-11-2025].

[57] swisskyrepo, "Github - swisskyrepo/payloadsallthethings: A list of useful payloads and bypass for web application security and pentest/ctf," https://github.com/swisskyrepo/PayloadsAllTheThings, 2019, [Accessed 09-04-2026].

[58] 0dayCTF, "Online - reverse shell generator," https://www.revshells.com/, 2021, [Accessed 09-04-2026].

[59] PyPI, "PyPI &xB7; The Python Package Index — pypi.org," https://pypi.org, 2025, [Accessed 10-11-2025].

[60] Nmap Project, "Ncat — nmap's netcat replacement," https://nmap.org/ncat/, 2025, accessed: 2025-11-10.

[61] "syscalls(2) - Linux manual page — man7.org," https://man7.org/linux/man-pages/man2/syscalls.2.html#:~:text=/usr/include/asm/unistd.h, 2025, [Accessed 30-10-2025].

[62] "colab.google," https://colab.google/, 2025, [Accessed 14-11-2025].

[63] C. Perry, "Colab Nvidia GPU," https://blog.tensorflow.org/2022/09/colabs-pay-as-you-go-offers-more-access-to-powerful-nvidia-compute-for-machine-learning.html, 2022, [Accessed 13-11-2025].

[64] Meta, "meta-llama/Llama-3.1-8B-Instruct," https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct, 2025, [Accessed 09-11-2025].

[65] O. AI, "Introducing GPT-5.2 — openai.com," https://openai.com/index/introducing-gpt-5-2/, 2025, [Accessed 29-01-2026].

[66] C. A. Ramezan, T. A. Warner, A. E. Maxwell, and B. S. Price, "Effects of training set size on supervised machine-learning land-cover classification of large-area high-resolution remotely sensed data," *Remote Sens. (Basel)*, vol. 13, no. 3, p. 368, Jan. 2021.

[67] D. Rajput, W.-J. Wang, and C.-C. Chen, "Evaluation of a decided sample size in machine learning applications," *BMC Bioinformatics*, vol. 24, no. 1, p. 48, Feb. 2023.

[68] J. Prusa, T. M. Khoshgoftaar, and N. Seliya, "The effect of dataset size on training tweet sentiment classifiers," in *2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA)*, 2015, pp. 96–102.

[69] J. Senanayake, H. Kalutarage, and M. O. Al-Kadri, "Android mobile malware detection using machine learning: A systematic review," *Electronics*, vol. 10, no. 13, 2021. [Online]. Available: https://www.mdpi.com/2079-9292/10/13/1606

[70] D. Ucci, L. Aniello, and R. Baldoni, "Survey of machine learning techniques for malware analysis," *Computers & Security*, vol. 81, pp. 123–147, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167404818303808

[71] P. Developers, "CPython: The python programming language," https://github.com/python/cpython, gitHub repository, accessed 2025-10-12.

[72] Y. Liu, J. Cao, C. Liu, K. Ding, and L. Jin, "Datasets for large language models: A comprehensive survey," 2024. [Online]. Available: https://arxiv.org/abs/2402.18041

[73] "Llama 3.1 release," https://ai.meta.com/blog/meta-llama-3-1/, 2024, [Accessed 09-11-2025].

[74] "Open-sourced training datasets for large language models (llms)," https://kili-technology.com/blog/9-open-sourced-datasets-for-training-large-language-models, [Accessed 16-04-2026].

[75] "gabejabe/bsidesSF-gordon-ramsey · Hugging Face — huggingface.co," https://huggingface.co/gabejabe/bsidesSF-gordon-ramsey, 2024, [Accessed 11-11-2025].

[76] P. Kassianik, B. Saglam, A. Chen, B. Nelson, A. Vellore, M. Aufiero, F. Burch, D. Kedia, A. Zohary, S. Weerawardhena *et al.*, "Llama-3.1-foundationai-securityllm-base-8b technical report," *arXiv e-prints*, pp. arXiv–2504, 2025.

[77] "Meta releases llamafirewall, an open-source defense a gainst ai hijacking — deeplearning.ai," https://www.deeplearning.ai/the-batch/meta-releases-llamafirewall-an-open-source-defense-against-ai-hijacking/, 2025, [Accessed 12-11-2025].

[78] C. Rondanini, B. Carminati, E. Ferrari, A. Kundu, and A. Gaudiano, "Malware detection at the edge with lightweight llms: A performance evaluation," *ACM Transactions on Internet Technology*.

[79] HuggingFace, "State of open source on hugging face: Spring 2026 — huggingface.co," https://huggingface.co/blog/huggingface/state-of-os-hf-spring-2026, 2026, [Accessed 09-04-2026].

[80] "GitHub - s2e-lab/hf-model-analyzer — github.com," https://github.com/s2e-lab/hf-model-analyzer.git, 2025, [Accessed 11-11-2025].

[81] P. Brown, A. Brown, M. Gupta, and M. Abdelsalam, "Online malware classification with system-wide system calls in cloud iaas," in *2022 IEEE 23rd International Conference on Information Reuse and Integration for Data Science (IRI)*. IEEE, 2022, pp. 146–151.

[82] "What is few shot prompting?" https://www.ibm.com/think/topics/few-shot-prompting, 2025, [Accessed 11-11-2025].

[83] "Slutterprime — pypi.org," https://pypi.org/project/slutterprime/, 2026, [Accessed 28-01-2026].

[84] "Dynahug-detector-artifact," https://dynahug-detector.github.io/, 2026, [Accessed 15-04-2026].